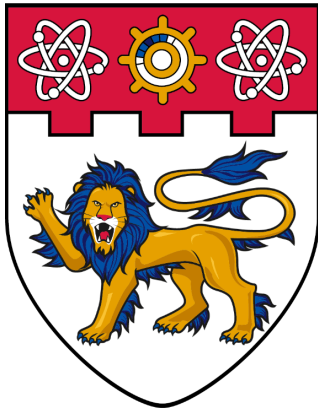




**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

COLLEGE OF COMPUTING AND DATA SCIENCE

SC3020 Database System Principles

Project 1
Group 7

Name	Matriculation Number
JASMINE TYE JIA WEN	U2321408J
PARK JIHAE	U2220524F
PONYUENYONG KRITCHANAT	U2322517E
SIH JIA QI	U2321900L
SUNKARA BHARGAVI	U2222578K

Code and Video Link :

<https://drive.google.com/drive/u/0/folders/1Uzw16eCcgtcbD4Fwyl9gSH7avlcUU2ma>

Table of Contents

1. Introduction.....	3
2. Storage Implementation.....	3
2.1 Overview.....	3
2.2 Record Structure.....	4
2.3 Block structure.....	5
2.4 Saving Records to Binary File.....	5
2.5 Buffer Manager.....	5
2.5.1 Fetching a Block.....	6
2.5.2 Buffer Manager Operations.....	6
3. B+ Tree Implementation.....	7
3.1 B+ Tree Structure.....	7
3.2 B+ Tree Construction.....	8
3.2.1 Initialization.....	8
3.2.2 Sequential Data Loading.....	8
3.2.3 Tree Balancing During Insertion.....	9
3.3 B+ Tree Deletion Implementation.....	9
3.3.1 B+ Tree Standard Deletion Implementation.....	9
3.3.1.1 Traversal and Key Collection.....	9
3.3.1.2 Individual Deletion with Rebalancing.....	10
3.3.2 B+ Tree “Lazy” Implementation.....	10
3.3.2.1 Pure Tree Traversal and Key Collection.....	10
3.3.2.2 Disk Marking.....	11
3.3.2.3 Tree Compaction & Rebalancing.....	11
3.3.2.3.1 Compaction.....	11
3.3.2.3.2 Rebalancing.....	12
3.3.2.3.3 Tree Rebalancing Operations.....	13
3.4 B+ Tree Storage.....	13
3.4.1 Serialization Architecture.....	13
3.4.2 Deserialization and Tree Reconstruction.....	14
3.4.3 Leaf Link Reconstruction.....	14
4. Results.....	14
4.1 Task 1: Storage Component Design and Data Organization.....	14
4.2 Task 2: Indexing Component and B+ Tree Construction.....	15
4.3 Task 3: Record Deletion and Performance Evaluation.....	16
4.3.1 B+ Tree Standard Deletion.....	16
4.3.2 Lazy B+ Tree Deletion.....	17
4.3.3 Linear Scan Deletion.....	17
4.3.4 Analysis and Comparison of Deletion Methods.....	18
5. Conclusion.....	20

1. Introduction

Database management systems (DBMS) play a key role in how data is stored, organized, and accessed. In this project, we delved into both the storage and indexing components of a DBMS by simulating a simple system. NBA game data was stored in binary files to mimic disk storage, while a B+ tree index was built to support efficient queries. We further explored query processing by comparing different deletion strategies, including linear scan and variations of B+ tree deletion, to better understand their impact on performance.

2. Storage Implementation

2.1 Overview

The dataset provided is the NBA games dataset from 2014 to 2022 with attributes as shown in the table below. The dataset contains a total of 26,651 rows, of which 99 rows have incomplete data.

Column Name	Description
GAME_DATE_EST	Game date in Eastern Standard Time format
TEAM_ID_home	Unique identifier for the home team
PTS_home	Total points scored by the home team
FG_PCT_home	Field goal percentage for home team (successful shots / total attempts)
FT_PCT_home	Free throw percentage for home team (successful free throws / total attempts)
FG3_PCT_home	Three-point field goal percentage for home team
AST_home	Number of assists by the home team
REB_home	Number of rebounds by the home team
HOME_TEAM_WINS	Indicates if the home team won (1) or lost (0)

Table 1: Attributes of the NBA Games Dataset (2014 to 2022)

While reading the data from the text file, we performed checks to ensure that all fields were properly filled. Any rows with missing values, incomplete fields, or invalid entries were skipped, resulting in the removal of 99 rows from the dataset.

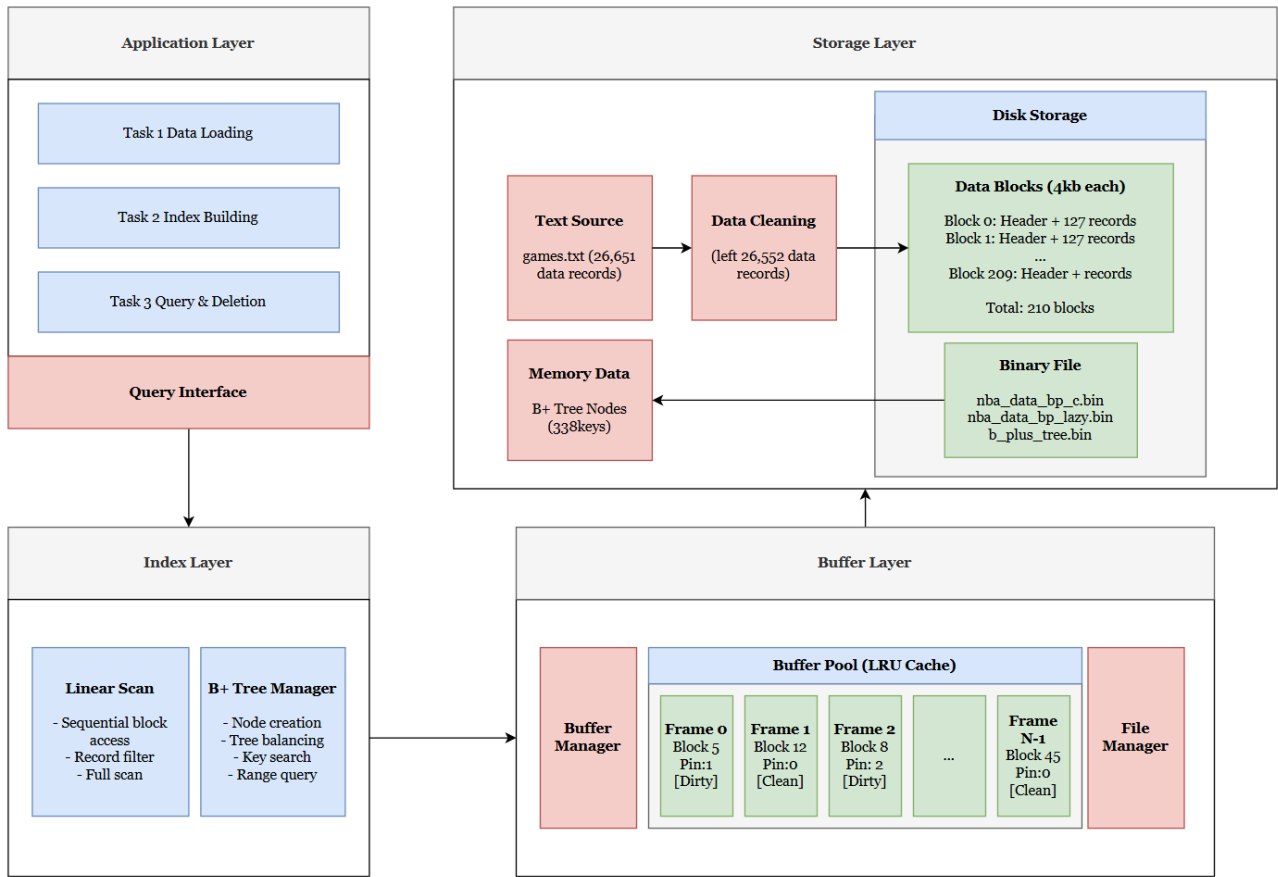


Figure 1: System Architecture for DBMS

2.2 Record Structure

The **NBA_RECORD** structure is designed to store NBA game data as compactly as possible. This is achieved by using smaller data types where appropriate - for example **AST_HOME** and **REB_HOME** are stored as short instead of int since their values are small.

The game date is represented in a 3-byte array **game_date_est[3]**, with 12 bits for the year, 4 bits for the month and 5 bits for the data, reducing storage compared to a standard 4-byte integer or date structure.

An **is_valid** flag is included as metadata to support lazy deletion, marking records as invalid without immediate block compaction. Additionally, **#pragma pack(push, 4)** is used to ensure that the structure is aligned on 4-byte boundaries, minimizing padding while maintaining efficient access.

Column Name	Data Type	Offset (bytes)
is_valid	bool	0-3
TEAM_ID_home	int	4-7
PTS_home	int	8-11
FG_PCT_home	float	12-15
FT_PCT_home	float	16-19
FG3_PCT_home	float	20-23
AST_home	short	24-25
REB_home	short	26-27
HOME_TEAM_WINS	bool	28
game_date_est	unsigned char[3]	29-31

Table 2: NBA_record fields with data types and offsets

The size of **NBA_Record** therefore spans 32 bytes, giving a compact, page-friendly record size.

2.3 Block structure

Each block contains:

- **block_id** : uniquely tags each page-sized buffer frame so the buffer manager can map cached pages back to their on-disk location
- **record_count** : tracks how many NBA_Record entries in the flexible **records[]** tail are currently meaningful
- **is_dirty** : lets the buffer know whether the page needs to be written back to disk when it is evicted, avoiding unnecessary I/O
- **timestamp** : stores the last modification time for LRU policy
- **records[]** : a flexible array that occupies all remaining bytes of the page with NBA_Record structs, giving a dense, variable-capacity payload per block

Blocks are designed to match the operation system's page size for optimal I/O performance.

```
// Block size determination
long get_block_size(void) {
    #ifdef _WIN32
        SYSTEM_INFO si;
        GetSystemInfo(&si);
        block_size = si.dwPageSize; // Windows page size
    #elif defined(__linux__) || defined(__unix__) || defined(__APPLE__)
        long res = sysconf(_SC_PAGESIZE); // Unix/Linux/macOS page size
        block_size = res;
    #endif
    return block_size;
}
```

At load time the program computes how many records fit :

```
size_t block_header_size = offsetof(Block, records);
int usable_space = block_size - (int) block_header_size;
int records_per_block = usable_space / sizeof(NBA_Record);
```

On a machine with 4KB memory pages, the block size is 4096 bytes. With the current packing, the block header is 16 bytes, and each NBA_Record is 32 bytes. Hence, storing $\frac{4096-16}{32} = 127$ records per block.

2.4 Saving Records to Binary File

The data is first read from the text file into memory, with dynamic allocation to hold all records. Once loaded, the records are sorted based on **FT_PCT_home**, which is used as the key for the B+ tree index. This ordering improves the efficiency of range queries and deletions that will be performed later. The sorted records are then grouped into blocks that fit within single memory pages and managed by the buffer manager.

This process resulted in a total of 210 blocks, which are written sequentially to a binary file with a size of 860,160 bytes. This binary file acts as our **simulated disk**, supporting efficient storage and indexed access.

2.5 Buffer Manager

A buffer manager was implemented to manage in-memory copies of data blocks, reducing the number of I/O operations to the simulated disk (binary file) and improving access performance. By maintaining a buffer pool of

recently used blocks in memory, it avoids repeated disk reads and provides faster access to frequently needed data. This reduces latency, improves efficiency and ensures that only necessary disk operations are performed.

The **BufferFrame** holds a single block of data along with metadata for management: the block itself, its ID in the binary file, a dirty flag to track modifications, a pin count to track active usage, and a timestamp for LRU replacement.

```
typedef struct BufferFrame{
    Block *block;    // Pointer to the actual block data
    int block_id;    // Block ID on disk
    bool is_dirty;    // Flag indicating if the block has been modified
    int pin_count;    // Number of clients currently using this block
    time_t last_used; // Timestamp of last access for LRU eviction
} BufferFrame;
```

The **BufferManager** struct keeps track of the buffer pool (frames array) along with metadata for management.

```
typedef struct BufferManager{
    BufferFrame *frames; // Array of buffer frames
    int num_frames;      // Number of frames in the buffer pool
    int block_size;      // Size of each block in bytes
    FILE *file;          // Pointer to the underlying binary file
} BufferManager;
```

2.5.1 Fetching a Block

To access a block, the **buffer_manager_get_block** method first checks whether the requested block is already present in the buffer pool. If the block is found, the corresponding frame is pinned to prevent eviction and its last-used timestamp is updated to reflect recent usage.

If the block is not present in memory, a victim frame is selected for replacement using the Least Recently Used (LRU) strategy, skipping any frames that are currently pinned. If the victim frame contains a dirty block, it is written back to the binary file before being overwritten.

The requested block is then read from the binary file (simulated disk) and loaded into the chosen frame, and the frame's metadata is updated. A small delay of 0.04 ms is added to simulate the time required for sequential disk I/O.

```
function buffer_manager_get_block(buffer_manager, block_id):
    if block_id in buffer_pool:
        increment pin_count
        update last_used timestamp
        return block
    else:
        victim = pick_LRU_victim()
        if victim is dirty:
            write victim.block to file
        load block from file into victim
        set pin_count = 1
        update last_used
        return block
```

2.5.2 Buffer Manager Operations

Other operations that the buffer manager provides for managing block states in memory are described briefly below.

- **buffer_manager_init** initializes the buffer pool with a specified number of frames and block size, opening the binary file.

- `buffer_manager_shutdown` ensures all dirty blocks are written back to the file and frees allocated memory.
- `buffer_manager_mark_dirty` flags a block as modified so it will be persisted during future flush operations.
- `buffer_manager_unpin` decreases a block's pin count, making it eligible for replacement.
- `buffer_manager_flush` writes all dirty blocks currently in memory back to the binary file to maintain data consistency. Together, these operations allow the buffer manager to efficiently handle in-memory copies of blocks while keeping the binary file synchronized.

3. B+ Tree Implementation

3.1 B+ Tree Structure

The B+ tree implementation uses a unified node structure `BPlusTreeNode` that efficiently handles both internal and leaf nodes through conditional field usage.

```
typedef struct BPlusTreeNode {
    float *keys;           // Array of keys stored in the node
    NBA_Record_Pointer *data; // Data pointers (leaf nodes only)
    int numKeys;           // Current number of keys in the node
    bool isLeaf;           // Node type identifier
    struct BPlusTreeNode **children; // Child pointers (internal nodes only)
    struct BPlusTreeNode *next; // Leaf node linking for range queries
} BPlusTreeNode;
```

The B+ Tree is indexed on `FT_PCT_HOME` which was chosen because it provides a good selectivity for range queries and supports the project's deletion operation efficiently.

The maximum number of keys per node (n) is dynamically calculated based on block size to optimize memory utilization:

```
int calculateMaxKeys(int block_size, bool isLeaf) {
    int fixed_size = sizeof(void *) * 4 + sizeof(int) + sizeof(bool);
    int per_key_size = isLeaf ? (sizeof(float) + sizeof(NBA_Record_Pointer))
                             : (sizeof(float) + sizeof(void *));
    if (block_size <= fixed_size)
        return 0;
    return (block_size - fixed_size) / per_key_size;
}
```

Fixed Overhead

- 4 pointers = 4×8 bytes = 32 bytes
- 1 integer = 1×4 bytes = 4 bytes
- 1 boolean = 1×1 byte = 1 byte
- Total Fixed Overhead = $32 + 4 + 1 = 37$ bytes

Per-Key Overhead

- Leaf Nodes: 12 bytes (4-byte float key + 8-byte NBA_Record_Pointer)
- Internal nodes: 12 bytes (4-byte float key + 8-byte child pointer)

$$MaxKey, N = \lfloor \frac{(blockSize - fixedOverhead)}{perKeyOverhead} \rfloor bytes$$

Figure 2: Maximum Number of Keys Per Node

For the system's 4KB blocks: $N = \lfloor \frac{4096 - 37}{12} \rfloor bytes = 338 bytes$

Internal Nodes store only keys and child pointers for navigation, with **data** set to NULL. The keys here act as routing information rather than actual data references.

Leaf Nodes store the actual index keys with corresponding data pointers. Each key maps to a pointer - **NBA_Record_Pointer**. These leaf nodes are linked horizontally via the next pointers for efficient sequential access.

The content of the **NBA_Record_Pointer** is as follows -

```
typedef struct NBA_Record_Pointer {
    int block_id; // which block the record is in
    int record_idx; // index of the record within the block
} NBA_Record_Pointer;
```

3.2 B+ Tree Construction

The B+ tree is built through the **Iterative Insertion** method.

3.2.1 Initialization

The **createBPlusTree()** function initializes a new B+ tree. It begins by calculating the maximum number of keys that can be stored in a node based on the block size using the **calculateMaxKeys()** function (as described in Section 3.1). After this, it allocates memory for a new **BPlusTree** structure and sets up the root node, which is initialized as a leaf.

```
MAX_KEYS = calculateMaxKeys(g_block_size, 0); // Sets N = 338
BPlusTree *tree = createBPlusTree(); // Creates empty tree with single leaf node
```

3.2.2 Sequential Data Loading

```
for (int b = 0; b < num_blocks; b++) {
    Block *blk = buffer_manager_get_block(bm, b);
    for (int r = 0; r < blk->record_count; r++) {
        NBA_Record *rec = &blk->records[r];
        if (rec->is_valid) {
            NBA_Record_Pointer ptr = {blk->block_id, r};
            insert(tree, rec->ft_pct_home, ptr); // Insert key-pointer pair
        }
    }
}
```

This data loading iterates through all 210 disk blocks sequentially, extracting valid records and inserting their (**FT_PCT_home**, record_pointer) pairs into the B+ tree. Since records are pre-sorted by **FT_PCT_home** from

Task 1, insertions occur in ascending order and benefit from cache locality which causes all insertions to occur at the rightmost leaf node. This results in 157 nodes.

3.2.3 Tree Balancing During Insertion

The insertion process maintains B+ tree properties through automatic node splitting when capacity constraints are violated. During leaf node insertion, new key-pointer pairs are inserted in sorted order within leaf nodes. When a leaf exceeds MAX_KEYS (338), overflow handling is triggered. The node splits into two nodes, where the left node retains $\lfloor \frac{n+1}{2} \rfloor$ keys and the right node receives $\lceil \frac{n+1}{2} \rceil$ keys. For $n = 338$, this results in a 169-170 split distribution to ensure both nodes satisfy the minimum occupancy constraint of $\lfloor \frac{n+1}{2} \rfloor = 169$ keys.

The first key of the newly created right node propagates upward as a separator key to the parent internal node to establish the boundary between the two child nodes. This separator insertion may trigger recursive splitting if the parent also exceeds its capacity, with the splitting process potentially cascading up to the root level. When the root itself overflows, a root split occurs where a new root node is created with the previous root becoming a child, thereby increasing the tree height by one level.

```
function insert(tree, key, value):
  if tree is empty:
    create a new root as a leaf node
    return

  leaf = findLeaf(tree.root, key)

  insertInLeaf(leaf, key, value);

  if leaf is overfull (numKeys > MAX_KEYS):
    (newLeaf, newKey) = splitLeaf(leaf)
    insertInParent(tree, leaf, newKey, newLeaf)
```

3.3 B+ Tree Deletion Implementation

Two B+ deletion approaches were implemented to compare performance, standard deletion and lazy deletion.

3.3.1 B+ Tree Standard Deletion Implementation

The `deleteKeysAboveThreshold` function removes all keys in the B+ tree whose `FT_PCT_home` values exceed 0.9. The deletion process is structured in two main phases: **key collection** and **key deletion**. This design ensures that the integrity of the B+ tree is safely maintained while performing multiple deletions efficiently.

3.3.1.1 Traversal and Key Collection

In the first phase, the tree is traversed to identify all keys with `FT_PCT_home` values above the threshold of 0.9. The traversal begins at the **leftmost leaf node** of the B+ tree and proceeds sequentially through the linked list of leaf nodes. Each leaf node is scanned for keys exceeding the threshold, and the qualifying keys are stored in the `keys_to_remove` array.

By collecting all target keys before performing any deletions, this approach ensures that the tree is traversed safely, without being affected by structural changes that may occur during deletion, such as node merges or redistributions.

```
// Traverse leaf chain to collect all keys above threshold
leaf ← leftmost_leaf(tree)
keys_to_remove ← []
```

```

while leaf ≠ null do
  for each key in leaf.keys do
    if key.value > threshold then
      append(keys_to_remove, key.value)
    end if
  end for
  leaf ← leaf.next
end while

```

3.3.1.2 Individual Deletion with Rebalancing

After all keys above the threshold have been collected, the function iterates through the `keys_to_remove` array and deletes each key individually using the `deleteKey` method. The `deleteKey` method locates the leaf containing the key by traversing through the tree from the root, marks the corresponding record as deleted (invalid) in the buffer, removes the key from the leaf node, and rebalances the tree by borrowing keys from siblings or merging nodes with siblings in case of underflow.

Because rebalancing may involve merging or redistributing keys among nodes, performing deletions once by one after collecting all keys prevents traversal issues, guarantees that no keys are skipped, and maintains the correctness of the tree structure throughout the operation.

Algorithm: Delete Keys Individually

Result: Delete keys individually and rebalance affected nodes

Function `deleteKeysIndividually (tree, keys_to_remove)`

```

foreach key in keys_to_remove do
  leaf ← findLeaf(tree.root, key);
  recordPtr ← locateRecordPointer(leaf, key);
  markRecordAsDeleted(bufferManager, recordPtr);
  removeKeyFromLeaf(leaf, key);
  if leaf.numKeys < minKeys then
    | rebalanceNode (tree, leaf);
  end
end
end

```

Figure 3: Individual Key Deletion Algorithm

Algorithm: Rebalance Node After Underflow

Result: Rebalance node after underflow, possibly propagating upward

Function `rebalanceNode (tree, node)`

```

if canBorrowFromSibling(node) then
  | borrowKeys(node, sibling);
end
else
  | mergeWithSibling(tree, node, sibling);
  | if parent.numKeys < minKeys then
    | | rebalanceNode (tree, parent);
  end
end
end

```

Figure 4: Node Rebalancing After Underflow Algorithm

3.3.2 B+ Tree “Lazy” Implementation

Lazy deletion optimizes bulk operations by deferring expensive B+ tree rebalancing until after all records that are supposed to be deleted are marked.

3.3.2.1 Pure Tree Traversal and Key Collection

The first phase of the B+ Tree “Lazy” Deletion performs a single sequential scan through the B+ tree’s leaf chain to collect all keys exceeding the threshold, along with their corresponding record pointers. This phase involves only tree traversal with no disk I/O operations.

Algorithm: Collect Keys for Lazy Deletion

Result: Collect all keys above threshold for deletion

Function collectKeysForLazyDeletion (*tree, threshold*)

```

keysToDelete ← empty array;
recordsToDelete ← empty array;
count ← 0;
leaf ← tree.root;
while leaf is not a leaf node do
    increment_index_nodes.accessed();
    leaf ← leaf.children[0];
end
while leaf ≠ null do
    increment_leaf_nodes.accessed();
    for i ← 0 to leaf.numKeys - 1 do
        if leaf.keys[i] ≥ threshold then
            keysToDelete[count] ← leaf.keys[i];
            recordsToDelete[count] ← leaf.data[i];
            count ← count + 1;
        end
    end
    leaf ← leaf.next;
end
return (keysToDelete, recordsToDelete, count);

```

Figure 5: Collect Keys for Lazy Deletion Algorithm

3.3.2.2 Disk Marking

The second phase marks the collected records as deleted on the disk setting their *is_valid* flag to false. It uses the record pointers collected in the previous phase, avoiding redundant tree searches. This phase does logical deletion instead of physical deletion, marking the flag here does not reclaim space immediately. The record remains physically present in the data block after the completion of this phase.

Algorithm: Mark Collected Keys as Deleted

Result: Mark collected keys as logically deleted in buffer manager

Function markCollectedKeysAsDeleted (*bufferManager, recordsToDelete, count*)

```

markedCount ← 0;
for k ← 0 to count - 1 do
    recordPtr ← recordsToDelete[k];
    block ← bufferManager.getBlock(recordPtr.block_id);
    if block ≠ null and recordPtr.record_idx ≠ block.record_count then
        block.records[recordPtr.record_idx].is_valid ← false;
        bufferManager.markDirty(block);
        bufferManager.unpin(block);
        markedCount ← markedCount + 1;
    end
end
return markedCount;

```

Figure 6: Mark Collected Keys as Deleted Algorithm

3.3.2.3 Tree Compaction & Rebalancing

After marking the records as deleted, the B+ tree index still contains stale entries - keys pointing to invalid records. This phase removes these entries and restores the tree's structural properties through a two-step process: compaction followed by rebalancing.

3.3.2.3.1 Compaction

Compaction is the process of physically removing deleted entries from leaf nodes to reclaim storage space and eliminate fragmentation. The algorithm uses an in-place two-pointer technique to compact arrays efficiently without requiring additional memory allocation. The compaction process is necessary as queries would skip over invalid pointers which wastes traversal time. Without compaction, the tree nodes would contain “dead” space which causes storage overhead.

Algorithm: Compact Deleted Entries

Result: Compact deleted entries in all leaf nodes

Function compactDeletedEntries (tree, threshold)

```

leaf ← tree.root;
while leaf is not a leaf node do
    increment_index_nodes_accessed();
    leaf ← leaf.children[0];
end
total_removed ← 0;
while leaf ≠ null do
    increment_leaf_nodes_accessed();
    originalKeys ← leaf.numKeys;
    writeldx ← 0;
    for readIdx ← 0 to leaf.numKeys - 1 do
        if leaf.keys[readIdx] ≤ threshold then
            if writeldx ≠ readIdx then
                leaf.keys[writeldx] ← leaf.keys[readIdx];
                leaf.data[writeldx] ← leaf.data[readIdx];
            end
            writeldx ← writeldx + 1;
        end
    end
    leaf.numKeys ← writeldx;
    total_removed ← total_removed + (originalKeys - writeldx);
    leaf ← leaf.next;
end
return total_removed;

```

Figure 7: Compact Deleted Entries Algorithm

3.3.2.3.2 Rebalancing

After compaction, leaf nodes may have too few nodes, violating B+ tree occupancy requirements. **Rebalancing** is the process of redistributing or merging nodes to restore these properties while maintaining tree balance - all leaves at the same depth. From the pseudocode below, multiple-rounds of rebalancing is done until convergence and no B+ minimum occupancy requirements are violated. The process of rebalancing could create cascading effects as shown in Figure 8.

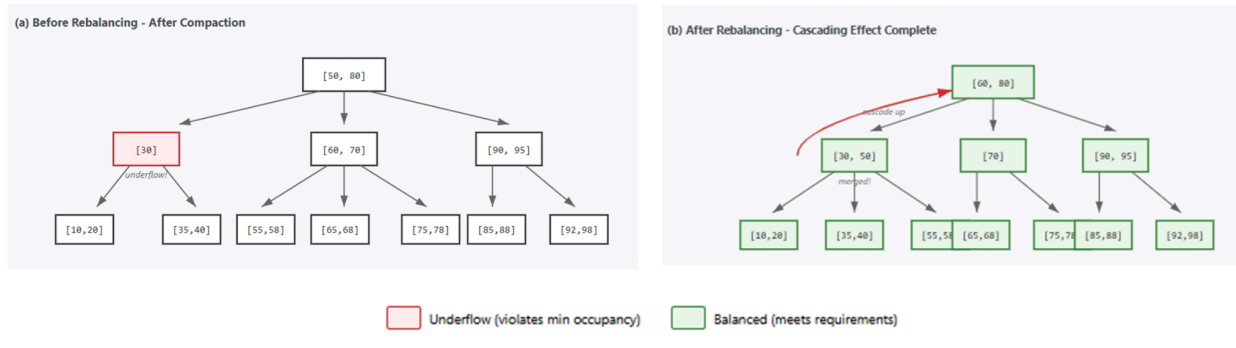


Figure 8: Cascading Effects of B+ Tree Rebalancing

Algorithm: Strict B+ Tree Rebalancing

Result: Strictly rebalance the B+ Tree until convergence

Function strictRebalance (tree)

```

rebalance_rounds ← 0;
max_rounds ← 10;
while rebalance_rounds < max_rounds do
    foundViolation ← false;
    rebalance_rounds ← rebalance_rounds + 1;
    leafMinKeys ← [(MAX_KEYS + 1)/2];
    leafViolations ← fixUnderfullLeafNodes(tree, leafMinKeys);
    internalMinKeys ← [MAX_KEYS/2];
    internalViolations ← fixUnderfullInternalNodes(tree, internalMinKeys);
    foundViolation ← leafViolations OR internalViolations;
    if not foundViolation then
        break ;
    end
end
if tree.root.numKeys = 0 and tree.root has single child then
    oldRoot ← tree.root;
    tree.root ← tree.root.children[0];
    freeNode(oldRoot);
end

```

Figure 9: Strict Tree Rebalancing Algorithm

3.3.2.3.3 Tree Rebalancing Operations

The B+ tree rebalancing process employs three fundamental operations to restore occupancy requirements after bulk deletion and compaction. These operations are applied systematically to address node underflow violations while maintaining tree balance and structural integrity.

	Redistribution (Borrowing)	Node Merging	Root Collapse
Condition	Sibling node possesses more than minimum required keys (has excess capacity to lend).	Both underfull node and sibling at or near minimum occupancy (sibling cannot lend without violating own requirements).	Root becomes empty (0 keys) with exactly one child after cascading internal merges.
Action	Transfer keys from sibling to underfull node through parent separator. For leaf nodes: direct key/data movement. For internal nodes: parent separator brought down, sibling key promoted up.	Combine an underfull node with a sibling into a single node. For leaves: copy all keys/pointers. For internal: bring down parent separator, transfer all children. Free merged node, update leaf linked list.	Replace root with its only child, promoting child to new root. Free old root nodes from memory.
Parent Update	Separator key modified to reflect new boundary between nodes. For leaf borrowing: separator becomes new minimum. For internal: separator replaced by promoted key.	Remove separator key dividing merged nodes and pointer to deleted node. Parent loses one key, potentially triggering parent underflow and cascading rebalancing.	Not applicable (root has no parent).

Table 3: Tree Rebalancing Operation Comparison

The rebalancing algorithm follows a systematic priority. It first attempts redistribution from the left sibling if one exists and contains keys exceeding the minimum threshold. If this is not possible, the algorithm proceeds to attempt redistribution from the right sibling. Should both borrowing attempts fail, and a left sibling is available, a merge with the left sibling is performed. If no left sibling exists, the algorithm merges with the right sibling, which applies only to leftmost children. Finally, if internal node merges propagate upward and result in an empty root, the root is collapsed to maintain tree balance.

3.4 B+ Tree Storage

3.4.1 Serialization Architecture

The serialization process transforms the in-memory B+ tree structure into a compact binary representation suitable for disk storage. The system employs a recursive depth-first serialization strategy that preserves the complete tree hierarchy and all structural relationships

Algorithm: Serialize Node
Result: Serialize a B+ Tree node recursively
Function SERIALIZE_NODE (<i>file, node</i>)
write node.isLeaf and node.numKeys to file;
write node.keys to file;
if node.isLeaf then
write node.dataPointers to file;
end
else
foreach child in node.children do
SERIALIZE_NODE (<i>file, child</i>);
end
end

Figure 10: Node Serialization Algorithm

3.4.2 Deserialization and Tree Reconstruction

The deserialization module restores the B+ tree structure from its binary representation. It reads the serialized file in the same order it was written, reconstructing each node recursively. The process ensures data integrity by validating key counts and node types during reconstruction.

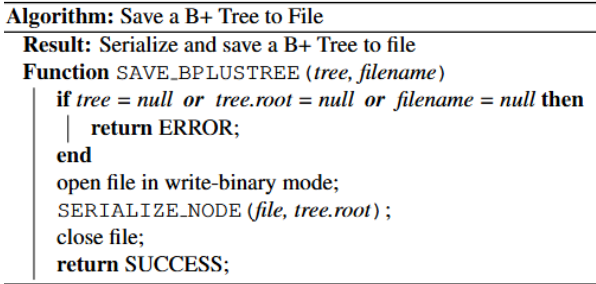


Figure 11: Save B+ Tree to File Algorithm

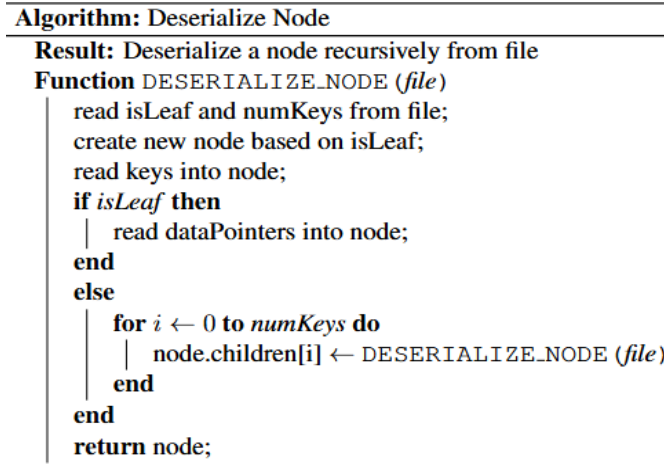


Figure 12: Node Deserialization Algorithm

3.4.3 Leaf Link Reconstruction

Since pointer addresses change between serialization and deserialization, the horizontal links between leaf nodes must be reconstructed during the loading process.

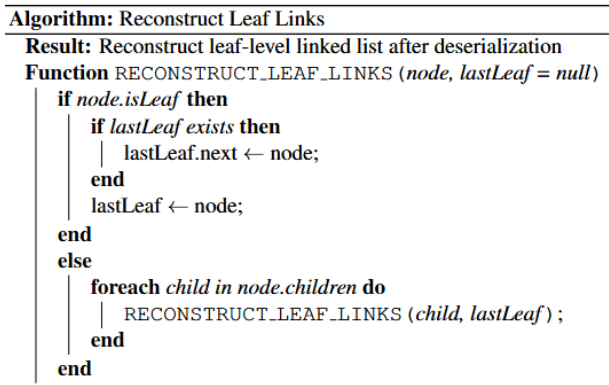


Figure 13: Leaf Link Reconstruction Algorithm

4. Results

4.1 Task 1: Storage Component Design and Data Organization

Storage Statistic	Value
Size of Record	32
Records per Block	127
Total Blocks	210
Total Records	26,552

Table 4: Task 1 Storage Statistics showing record size, records per block, total blocks, and total records.

Table 4 shows the storage statistics for the NBA game dataset after binary serialization. The implementation stores 26,552 game records across 210 disk blocks where each record occupies 32 bytes and each block contains 127 records.

Record Size The 32 bytes result from using the 4-byte alignment. This includes two boolean fields (2 bytes), two integer fields (8 bytes), three floating-point fields (12 bytes), two short integer fields (4 bytes), and a packed 3-byte date representation. To maintain the 4-byte alignment, the compiler inserts 3 bytes of padding to yield exactly 32 bytes per record.

Records per Block The 127 records is determined by the system page size of 4096 bytes minus 16 bytes of block metadata (Block ID, record count, and timestamp), resulting in 4080 bytes for data storage. Then, we compute $\lfloor \frac{4080}{32} \rfloor$ which yields 127 records per block.

Blocks To store all records in our database, we compute $\lceil \frac{26552}{127} \rceil$ which results in a total of 210 blocks.

4.2 Task 2: Indexing Component and B+ Tree Construction

Parameter	Value(s)
Max keys per node (n)	338
Number of nodes	157
Number of levels	2
Average of leaf keys	0.760
Root node keys	0.474, 0.500, 0.533, 0.550 0.560, 0.571, 0.579, 0.588 0.600, 0.600, 0.609, 0.611 0.618, 0.621, 0.625, 0.632 0.634, 0.639, 0.643, 0.647 0.650, 0.652, 0.655, 0.658 0.667, 0.667, 0.667, 0.667 0.667, 0.677, 0.680, 0.682 0.684, 0.686, 0.688, 0.692 0.692, 0.696, 0.697, 0.700 0.700, 0.704, 0.706, 0.708 0.710, 0.714, 0.714, 0.714 0.714, 0.720, 0.721, 0.722 0.724, 0.727, 0.727, 0.731 0.733, 0.733, 0.735, 0.737 0.739, 0.741, 0.741, 0.743 0.750, 0.750, 0.750, 0.750 0.750, 0.750, 0.750, 0.758 0.759, 0.760, 0.762, 0.762 0.765, 0.765, 0.767, 0.769 0.769, 0.773, 0.773, 0.774 0.778, 0.778, 0.778, 0.783 0.783, 0.786, 0.786, 0.788 0.789, 0.789, 0.792, 0.793 0.800, 0.800, 0.800, 0.800 0.800, 0.800, 0.806, 0.808 0.810, 0.810, 0.813, 0.813 0.815, 0.818, 0.818, 0.821 0.824, 0.824, 0.826, 0.828 0.833, 0.833, 0.833, 0.833 0.833, 0.846, 0.842, 0.842 0.846, 0.846, 0.850, 0.852 0.853, 0.857, 0.857, 0.860 0.864, 0.867, 0.867, 0.870 0.875, 0.875, 0.880, 0.882 0.885, 0.889, 0.892, 0.895 0.900, 0.905, 0.909, 0.913 0.917, 0.925, 0.933, 0.941 0.947, 0.958, 1.000

Table 5: B+ Tree Structure Statistics showing node configuration and root node key distribution

Table 5 shows the parameters of the B+ tree index built on the [FT_PCT_home](#) attribute consisting of a 2-level structure with 157 nodes, where each node can contain up to 338 keys

Maximum Keys per Node Each internal node stores pointers to child nodes (8 bytes) along with separator keys (4 bytes), requiring 12 bytes per entry. After accounting for the fixed overhead of 37 bytes, we compute $\lfloor \frac{4096 - 37}{12} \rfloor$ which results in 338 maximum keys per node.

Total Nodes Each leaf node must maintain at least $\lceil \frac{n+1}{2} \rceil = \lceil \frac{339}{2} \rceil = 170$ keys. With 26,552 records, the minimum number of leaf nodes required is $\lceil \frac{26,552}{170} \rceil = 157$ leaves. Therefore, the tree has 156 leaf nodes (level 1) with an average occupancy of 170 keys per leaf, plus 1 root node (level 0). Since 156 leaf nodes is less than the maximum 339 children an internal node can contain ($n + 1 = 338 + 1$), all leaves can be indexed by a single root node.

4.3 Task 3: Record Deletion and Performance Evaluation

4.3.1 B+ Tree Standard Deletion

Parameter	Value(s)
Max keys per node (n)	338
Number of nodes	146
Number of levels	2
Average of leaf keys	0.748
Root node keys	0.474, 0.500, 0.533, 0.550 0.560, 0.571, 0.579, 0.588 0.600, 0.600, 0.609, 0.611 0.618, 0.621, 0.625, 0.632 0.634, 0.639, 0.643, 0.647 0.650, 0.652, 0.655, 0.658 0.667, 0.667, 0.667, 0.667 0.667, 0.677, 0.680, 0.682 0.684, 0.686, 0.688, 0.692 0.692, 0.696, 0.697, 0.700 0.700, 0.704, 0.706, 0.708 0.710, 0.714, 0.714, 0.714 0.714, 0.720, 0.721, 0.722 0.724, 0.727, 0.727, 0.731 0.733, 0.733, 0.735, 0.737 0.739, 0.741, 0.741, 0.743 0.750, 0.750, 0.750, 0.750 0.750, 0.750, 0.750, 0.758 0.759, 0.760, 0.762, 0.762 0.765, 0.765, 0.767, 0.769 0.769, 0.773, 0.773, 0.774 0.778, 0.778, 0.778, 0.783 0.783, 0.786, 0.786, 0.788 0.789, 0.789, 0.792, 0.793 0.800, 0.800, 0.800, 0.800 0.800, 0.800, 0.806, 0.808 0.810, 0.810, 0.813, 0.813 0.815, 0.818, 0.818, 0.821 0.824, 0.824, 0.826, 0.828 0.833, 0.833, 0.833, 0.833 0.833, 0.840, 0.842, 0.842 0.846, 0.846, 0.850, 0.852 0.853, 0.857, 0.857, 0.860 0.864, 0.867, 0.867, 0.870 0.875, 0.875, 0.880, 0.882 0.885, 0.889, 0.892, 0.895

Metric	Value
Traversal Time (ms)	0.051
Deletion + Tree Rebalancing Time (ms)	2.972
Total Running Time (ms)	3.023
Number of Internal Nodes Accessed	2820
Number of Leaf Nodes Accessed	2991
Total Number of Nodes Accessed	5811
Number of Data Blocks Accessed	15
Number of Games Deleted	1778
Average of ft pct home	0.748

Table 6: B+ Tree Standard Deletion Result Statistics

The retrieval time for keys exceeding 0.9 is only 0.051 ms, as all relevant B+ tree nodes are in memory and no data block access is required (therefore no disk access is required).

During this process, 2820 internal nodes and 2991 leaf nodes were accessed, resulting in a total of 5811 nodes. The total node count is high because each key deletion involves traversing from the root to the leaf, causing internal nodes to be visited multiple times. Additionally, leaf and internal nodes are accessed repeatedly during key collection and individual deletions, including rebalancing operations such as borrowing or merging.

Only 15 data blocks are accessed because the records are stored sequentially in increasing order of **FT_PCT_home**. As a result, all records with **FT_PCT_home** values exceeding 0.9 are clustered together in a small subset of blocks. This means that out of the total 290 data blocks, only these contiguous blocks need to be read, allowing highly efficient access to the relevant records without scanning the entire dataset.

4.3.2 Lazy B+ Tree Deletion

		Parameter	Value(s)
		Max keys per node (n)	338
		Number of nodes	146
		Number of levels	2
		Average of leaf keys	0.748
		Root node keys	0.474, 0.500, 0.533, 0.550 0.560, 0.571, 0.579, 0.588 0.600, 0.600, 0.609, 0.611 0.618, 0.621, 0.625, 0.632 0.634, 0.639, 0.643, 0.647 0.650, 0.652, 0.655, 0.658 0.667, 0.667, 0.667, 0.667 0.667, 0.677, 0.680, 0.682 0.684, 0.686, 0.688, 0.692 0.692, 0.696, 0.697, 0.700 0.700, 0.704, 0.706, 0.708 0.710, 0.714, 0.714, 0.714 0.714, 0.720, 0.721, 0.722 0.724, 0.727, 0.727, 0.731 0.733, 0.733, 0.735, 0.737 0.739, 0.741, 0.741, 0.743 0.750, 0.750, 0.750, 0.750 0.750, 0.750, 0.750, 0.758 0.759, 0.760, 0.762, 0.762 0.765, 0.765, 0.767, 0.769 0.769, 0.773, 0.773, 0.774 0.778, 0.778, 0.778, 0.783 0.783, 0.786, 0.786, 0.788 0.789, 0.789, 0.792, 0.793 0.800, 0.800, 0.800, 0.800 0.800, 0.800, 0.806, 0.808 0.810, 0.810, 0.813, 0.813 0.815, 0.818, 0.818, 0.821 0.824, 0.824, 0.826, 0.828 0.833, 0.833, 0.833, 0.833 0.833, 0.840, 0.842, 0.842 0.846, 0.846, 0.850, 0.852 0.853, 0.857, 0.857, 0.860 0.864, 0.867, 0.867, 0.870 0.875, 0.875, 0.880, 0.882 0.885, 0.889, 0.892, 0.895
Metric	Value		
Phase 1: Pure Traversal Time (ms)	0.066		
Phase 2: Disk Marking Time (ms)	0.754		
Phase 3: Compaction Time (ms)	0.057		
Total Running Time (ms)	0.878		
Number of Internal Nodes Accessed	43		
Number of Leaf Nodes Accessed	3215		
Total Number of Nodes Accessed	3258		
Number of Data Blocks Accessed	15		
Number of Games Deleted	1778		
Average of ft pct home	0.748		

Table 7: Lazy B+ Tree Deletion Result Statistics

The lazy deletion algorithm completed in 0.878 ms, processing 1,778 deletions across three sequential phases: traversal (0.066 ms), disk marking (0.754 ms), and compaction (0.057 ms).

Phase 1 (traversal) accounts for only 7.5% of total execution time. The algorithm scans through leaf nodes using the B+ tree's horizontal linked list, collecting keys that exceed the threshold without modifying tree structure.

Phase 2 (disk marking) accounts for 85.9% of execution time, taking 0.754 ms to mark records as deleted on disk. The algorithm writes to 15 data blocks where records with `FT_PCT_home` > 0.9 are located. This is due to the sorted organization from Task 1, which clusters high free-throw percentage games in the rightmost blocks of the dataset. The disk marking phase represents the primary I/O bottleneck of the algorithm.

Phase 3 (compaction) performs tree restructuring in 0.057 ms (6.5% of total time). Rather than rebalancing after each deletion, the algorithm defers all structural updates until this final phase. The compaction removes deleted entries from leaf nodes and triggers rebalancing only for nodes that fall below minimum occupancy thresholds. This batch approach reduces the number of rebalancing operations compared to per-deletion maintenance.

After deletion, the tree contains 146 nodes with an average leaf key value of 0.748, down from the original 0.760. This is due to the removal of the `FT_PCT_home` values > 0.9 from the dataset. The 2-level structure still remains as the resulting node count still fits within a single root's capacity of 339 children.

4.3.3 Linear Scan Deletion

For the linear scan deletion, the algorithm linearly scans all the blocks. Each block is accessed via the buffer manager. Each record in a block is checked, and if it is valid and its `FT_PCT_home` exceeds 0.9, it is marked as deleted. After processing each block, the block is marked as dirty and unpinned.

```

for block_id from 0 to num_blocks - 1 do
  blk ← buffer_manager_get_block(bm, block_id)
  if blk = null then

```

```

    continue
end if

// Traverse all records in the block
for record_idx from 0 to blk.record_count - 1 do
    rec ← blk.records[record_idx]
    if rec.is_valid = true and rec.ft_pct_home > 0.9 then
        rec.is_valid ← false      // Mark record as deleted
    end if
end for

// Mark block as dirty and unpin
buffer_manager_mark_dirty(bm, blk)
buffer_manager_unpin(bm, blk)
end for

```

Metric	Value
Total Running Time (ms)	12.008
Number of Data Blocks Accessed	210

Table 8: Linear Scan Deletion Result

The linear scan takes a considerable amount of time (with an added 0.04 ms to simulate disk access) because it repeatedly performs disk I/O to load each block into the buffer pool. As a result, all 210 data blocks are accessed during the process.

4.3.4 Analysis and Comparison of Deletion Methods

The experimental evaluation reports the following key statistics:

Method	Traversal Time (ms)	Total Time (ms)	Internal Nodes	Leaf Nodes	Data Blocks
Standard B+ Tree	0.051	3.023	2820	2991	15
Lazy B+ Tree	0.066	0.878	43	3215	15
Linear Scan (Brute-force)	N/A	12.008	N/A	N/A	210

Table 9: Performance Comparison of Deletion Methods Implemented

Standard B+ Tree Traversal Time refers to the retrieval phase that collects all keys > 0.9 through sequential leaf traversal using horizontal links. This time represents pure tree navigation without any deletion operations or disk I/O for record marking. The efficiency stems from accessing tree nodes already cached in memory during the collection phase.

Lazy B+ Tree Traversal Time is slightly higher due to the additional overhead of collecting both keys and their corresponding record pointers in dynamic arrays. The algorithm must store `NBA_Record_Pointer` structures alongside key values, requiring more memory operations and occasional array resizing during collection.

The minimal difference between Lazy and standard B+ traversal times confirms that both methods achieve comparable efficiency for the key identification phase. This validates that the lazy approach's performance advantage comes primarily from optimized deletion processing rather than traversal improvements.

Standard B+ Tree Deletion accesses 2,820 internal nodes due to individual navigation for each of the 1,778 deletions. Every `deleteKey()` performs a full root-to-leaf traversal, repeatedly revisiting internal nodes. Additional accesses occur during rebalancing when underflows trigger borrowing or merging. A total of 2,991 leaf node accesses result from navigating to the specific leaf for each deletion. After key removal, rebalancing may access neighboring leaves to verify occupancy or redistribute keys. The high leaf count reflects repeated visits to the same leaves—each leaf containing multiple target keys is accessed once per deletion rather than once overall.

Lazy B+ Tree Deletion accesses only 43 internal nodes, mostly during the initial descent to the leftmost leaf (Phase 1) and the final rebalancing phase (Phase 3). The bulk collection phase removes the need for repeated navigation. It accesses 3,215 leaf nodes but visits each leaf exactly once in a sequential scan through the horizontal leaf chain. The algorithm traverses from the leftmost leaf using next pointers to collect all keys > 0.9 , scanning all leaves instead of jumping directly to targets. Although the leaf count is higher, this single comprehensive pass is more efficient because redundant traversals are eliminated. Minimal additional accesses occur during Phase 3 compaction since most structural work follows marking.

Standard deletion performs repeated targeted searches (fewer leaves but high repetition), while lazy deletion performs a single full scan (more leaves but no repetition). Despite scanning more leaves, the lazy approach achieves better performance by drastically reducing internal node accesses (43 vs 2,820) and avoiding redundant traversals and rebalancing operations.

Linear Scan Deletion brute force method bypasses the B+ tree index structure, accessing data blocks sequentially without any tree navigation. It does not distinguish between traversal and processing phases, as it performs both operations simultaneously during the sequential block scan.

B+ Tree Methods Data Block accesses only 15 data blocks due to the clustering effect of sorted data. Records with `FT_PCT_home` > 0.9 are stored contiguously because the initial data loading sorted records by this attribute. This spatial locality allows efficient access to only the relevant subset of the 210 total blocks.

Linear Scan Data Block must scan all 210 data blocks sequentially to locate target records, regardless of their actual distribution.

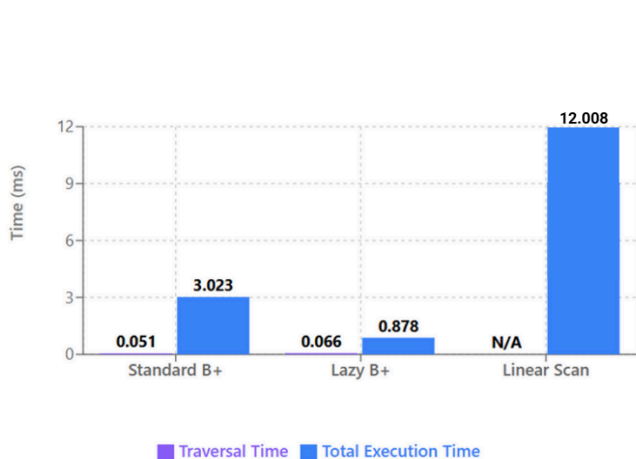


Figure 14: Deletion Running Time Comparison

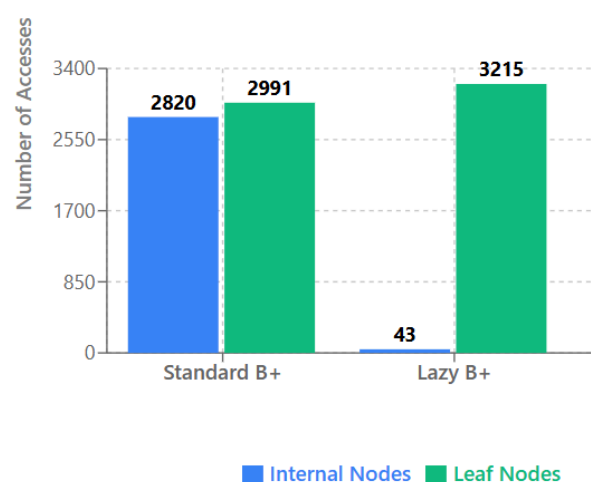


Figure 15: B+ Tree Node Access Count

5. Conclusion

This project implemented a database storage and indexing system for NBA game records and has shown significant performance improvements through proper data organization and algorithmic design. The storage component achieved high space utilization through compact record formatting and efficient block packing. The B+ tree index provided faster searches compared to sequential scanning. Lazy deletion proved to be more efficient than standard deletion methods by deferring tree maintenance and processing deletions in batches. Our analysis and results demonstrated how foundational database concepts storage, tree-based indexing, and batch processing can produce efficient and scalable query processing systems.